CHULETA COMPLETA · DESPLIEGUE WEB

MP_0614 · UD1 · Examen Modelo A · Preguntas 1, 2 y 3

Objetivo del examen: crear una web estática sobre tu CV, desplegarla en Tomcat; crear una aplicación que convierta l/100 km a mpg y calcule el impuesto; y enlazar ambas actividades.

MAPA GENERAL DEL EXAMEN

PREGUNTA 1	PREGUNTA 2	PREGUNTA 3
Web estática **HTML + CSS + imagen**	**Aplicación dinámica** **Formulario + cálculo + impuesto**	**Integración** **Enlace desde la web estática**
Se publica en Tomcat	Se ejecuta en Tomcat	La web CV permite abrir la aplicación

CONCEPTOS QUE DEBES DISTINGUIR

Elemento	Qué hace
Eclipse for Web Developers	Entorno de desarrollo donde creas los proyectos y archivos.
Apache Tomcat	Servidor que recibe peticiones HTTP y sirve/ejecuta aplicaciones web.
localhost	Tu propio ordenador.
Puerto 8080	Puerto habitual de Tomcat (puede cambiar según la configuración).
Desplegar	Publicar una aplicación en el servidor para poder acceder a ella desde el navegador.
Contexto	Nombre/ruta base de la aplicación dentro del servidor, por ejemplo /MiWeb.

PREGUNTA 1 · CREAR Y DESPLEGAR UNA WEB ESTÁTICA

Qué pide: una única página relacionada con tu CV, hecha preferentemente con HTML, que incluya una imagen y esté desplegada en Tomcat. Debes entregar una captura de la web con la URL y otra de la carpeta desplegada, además de explicar los pasos y cómo inicializas Tomcat.

1. Preparar el entorno

Necesitas Eclipse for Web Developers y Apache Tomcat instalado (el documento de ejemplo utiliza Tomcat 9.0). En Eclipse, la vista de servidores se abre desde Window → Show View → Servers.

2. Crear el proyecto Static Web Project

File → New → Other... → Web → Static Web Project → Next. Escribe un nombre, por ejemplo MiCV, y pulsa Finish. Según el procedimiento descrito en tu examen, los archivos web se crean dentro de la carpeta public.

3. Estructura de archivos recomendada

```
MiCV/
└── public/
    ├── index.html
    ├── estilos.css
    └── img/
        └── foto.jpg
```

4. Ejemplo de página HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Mi CV</title>
  <link rel="stylesheet" href="estilos.css">
</head>
<body>
  <h1>Mi currículum</h1>
  <p>Estudiante de Desarrollo de Aplicaciones Web</p>
  <img src="img/foto.jpg" alt="Imagen de presentación">
</body>
</html>
```

Ejemplo de CSS

```
body {
  font-family: Arial, sans-serif;
  background: #f1f5f9;
  text-align: center;
}
h1 { color: #2563eb; }
img { max-width: 250px; }
```

5. Configurar Tomcat en Eclipse

En la vista Servers, pulsa el enlace para crear un servidor o clic derecho → New → Server. Selecciona Apache → Tomcat v9.0 Server (o tu versión). Indica la carpeta donde instalaste Tomcat y pulsa Finish.

6. Iniciar y desplegar

En Servers, clic derecho sobre Tomcat → Start. Comprueba que aparece como Started. Después, clic derecho sobre el proyecto → Run As → Run on Server. Selecciona el servidor Tomcat y pulsa Finish.

7. Comprobar en el navegador

```
http://localhost:8080/MiCV/index.html
```

La URL se compone de protocolo + servidor + puerto + contexto del proyecto + archivo. Si el contexto tiene otro nombre, sustituye MiCV por el que corresponda.

8. Localizar la carpeta desplegada

En un despliegue directo, Tomcat suele utilizar la carpeta webapps. En el ejemplo del examen aparece:

```
C:\Program Files\Apache Software Foundation\Tomcat 9.0\webapps\agoodshop
```

Importante: si Eclipse utiliza un servidor Tomcat administrado por WTP, puede desplegar en una carpeta del workspace, como wtpwebapps. Comprueba la configuración del servidor en Eclipse; no asumas que siempre se usa la carpeta webapps de la instalación.

Capturas de la pregunta 1

Captura 1: página abierta en el navegador, imagen visible y URL completa en la barra de direcciones.
Captura 2: carpeta desplegada mostrando HTML, CSS e imagen. Añade una breve descripción del despliegue y de cómo iniciaste Tomcat.

PREGUNTA 2 · CONVERSOR L/100 KM → MPG + IMPUESTO

Qué pide: crear un servlet o cualquier otra aplicación ejecutable en Tomcat. El usuario introduce un consumo decimal en litros por cada 100 km, pulsa un botón y la aplicación muestra su conversión a mpg. Después debe calcular el impuesto según la tabla del enunciado.

1. Fórmula de conversión

El enunciado da esta fórmula:

mpg = 235,214 ÷ (consumo en l/100 km)

Ejemplo solicitado: para 8 l/100 km:

mpg = 235,214 / 8 = 29,40175 ≈ 29,40 mpg

Cuanto menor es el consumo en l/100 km, mayor es el valor mpg (mejor eficiencia).

OJO: discrepancia dentro del propio PDF

La fórmula matemática impresa es 235,214 dividido entre el consumo. Sin embargo, el fragmento de código de la página 6 muestra 235.214/(cons/100). Esa expresión divide otra vez el consumo entre 100 y no coincide con la fórmula del enunciado. Para los cálculos y ejemplos de esta chuleta se sigue la fórmula matemática impresa: 235.214 / cons.

2. Formulario HTML básico

```
<form action="ConsumoServlet" method="post">
  <label>Consumo (l/100 km):</label>
  <input type="number" name="consumo"
        step="any" min="0" required>
  <button type="submit">Calcular</button>
</form>
```

El atributo name="consumo" permite recuperar el valor en el servlet. step="any" permite introducir decimales.

3. Flujo de funcionamiento

PASO	QUÉ OCURRE
1. Usuario	Introduce el consumo (por ejemplo, 8).
2. Formulario	Envía el dato al servlet mediante POST.
3. Servlet	Lee el parámetro consumo y lo convierte a número decimal.
4. Cálculo	Calcula mpg = 235.214 / consumo.
5. Respuesta	Muestra el resultado en el navegador.

4. Ejemplo de servlet (Tomcat 9)

Este ejemplo utiliza la API javax.servlet, habitual en Tomcat 9. En versiones más nuevas como Tomcat 10+, los imports cambian a jakarta.servlet.

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/ConsumoServlet")
public class ConsumoServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
                          throws ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");

        double consumo = Double.parseDouble(
            request.getParameter("consumo")
        );

        double mpg = 235.214 / consumo;

        PrintWriter out = response.getWriter();
        out.println("<h1>Resultado</h1>");
        out.println("<p>Consumo: " + consumo + " l/100 km</p>");
        out.println("<p>Equivalencia: " + mpg + " mpg</p>");
    }
}
```

Es un ejemplo didáctico mínimo: en una aplicación completa conviene validar que el valor sea numérico y mayor que cero, y presentar errores comprensibles si el usuario introduce datos incorrectos.

5. Tabla del impuesto (según el enunciado)

MPG	Impuesto ($)
22,5 o más	0 $
17,5 – 22,4	2.000 $
12,5 – 17,4	4.000 $
12,4 o menos	7.700 $

La tabla clasifica el vehículo según el mpg calculado: un mpg alto significa más eficiencia y menor impuesto; un mpg bajo significa peor eficiencia y mayor impuesto.

6. Lógica Java para calcular el impuesto

```
double calcularImpuesto(double mpg) {
    if (mpg >= 22.5) {
        return 0;
    } else if (mpg >= 17.5) {
        return 2000;
    } else if (mpg >= 12.5) {
        return 4000;
    } else {
        return 7700;
    }
}
```

La última rama cubre los valores inferiores a 12,5 mpg. La tabla del enunciado expresa el límite inferior como 12,4; los intervalos están escritos con una cifra decimal.

7. Dos ejemplos obligatorios del examen

Consumo	Cálculo mpg	Tramo de la tabla	Impuesto
4 l/100 km	235,214 / 4 = 58,80	22,5 o más	0 $
20 l/100 km	235,214 / 20 = 11,76	12,4 o menos	7.700 $

También debes mostrar el ejemplo de 8 l/100 km solicitado en el apartado de conversión: 235,214 / 8 = 29,40 mpg.

8. Capturas que pide la pregunta 2

Conversión: dos capturas con la barra de direcciones de Tomcat: pantalla del formulario y resultado al convertir 8 l/100 km (aprox. 29,40 mpg). Explica brevemente cómo creaste y ejecutaste la aplicación.
Impuesto: muestra dos ejecuciones diferentes: 4 l/100 km (0 $) y 20 l/100 km (7.700 $).

PREGUNTA 3 · CONECTAR LAS DOS ACTIVIDADES

Qué pide: que la aplicación creada en la pregunta 2 sea accesible desde el sitio web de la pregunta 1. Debes aportar capturas de la nueva funcionalidad y una breve descripción de los pasos.

1. Idea de la integración

La web estática del CV actúa como página principal. Desde ella añades un enlace que lleva a la aplicación de conversión/impuesto desplegada en Tomcat.

CV / página principal (Pregunta 1)
↓ enlace
Aplicación de consumo (Pregunta 2)
↓
Conversión a mpg + cálculo del impuesto

2. Añadir un enlace en index.html

Si el contexto de la aplicación dinámica es ConsumoApp y el servlet está mapeado como /ConsumoServlet, puedes añadir:

```
<nav>
  <a href="/ConsumoApp/">Calculadora de consumo</a>
</nav>
```

Si la página principal del conversor está en un archivo concreto, enlaza directamente a él. Por ejemplo:

```
<a href="/ConsumoApp/index.html">
  Abrir conversor de consumo
</a>
```

Si quieres enlazar directamente al servlet, el enlace sería:

```
<a href="/ConsumoApp/ConsumoServlet">
  Abrir calculadora
</a>
```

Usa la ruta real de tu proyecto y el mapeo real del servlet. Los nombres ConsumoApp e index.html son ejemplos, no nombres obligatorios.

3. Comprobar la integración

1. Inicia Tomcat.
2. Comprueba que las dos aplicaciones están publicadas en el mismo servidor.
3. Abre la web del CV en el navegador.
4. Pulsa el enlace nuevo.
5. Comprueba que se abre el conversor y que puedes realizar los cálculos.

4. Capturas que pide la pregunta 3

Haz las capturas necesarias para demostrar el recorrido completo: página principal del CV con el enlace visible, aplicación abierta después de pulsarlo y, si es necesario, resultado de una conversión/cálculo. Acompáñalas con una breve explicación.

PROBLEMAS FRECUENTES EN TODO EL EXAMEN

Problema	Qué comprobar
Tomcat no arranca	Que no haya otro proceso usando el puerto (por ejemplo, otro Tomcat iniciado desde XAMPP). Revisa la consola de Eclipse.
localhost:8080 no responde	Que el servidor esté en estado Started y que el puerto configurado sea realmente 8080.
Error 404	Revisa el contexto del proyecto, la ruta solicitada, el nombre del archivo y el mapeo del servlet.
No se ve el CSS o la imagen	Revisa las rutas relativas y que los archivos estén dentro de la carpeta publicada.
El servlet no reconoce el parámetro	Comprueba que el atributo name del input coincide con getParameter("...").
Error de imports javax/jakarta	Tomcat 9 usa javax.servlet; Tomcat 10+ usa jakarta.servlet. Comprueba que la versión y las librerías coinciden.
No abre el enlace entre proyectos	Comprueba el contexto real de la segunda aplicación y la ruta del archivo o servlet.

CHECKLIST FINAL · ANTES DE ENTREGAR

☐ Pregunta 1: web de una página relacionada con mi CV.

☐ Pregunta 1: imagen visible y CSS aplicado.

☐ Pregunta 1: web desplegada en Tomcat y URL visible.

☐ Pregunta 1: captura de la carpeta desplegada y explicación de los pasos.

☐ Pregunta 2: formulario acepta consumo decimal.

☐ Pregunta 2: conversión de 8 l/100 km mostrada (≈ 29,40 mpg).

☐ Pregunta 2: impuesto calculado según la tabla.

☐ Pregunta 2: ejemplo de 4 l/100 km (0 $).

☐ Pregunta 2: ejemplo de 20 l/100 km (7.700 $).

☐ Pregunta 3: enlace desde la web del CV a la aplicación.

☐ Pregunta 3: capturas que demuestran el recorrido y la funcionalidad.

RESUMEN ULTRARRÁPIDO PARA MEMORIZAR

P1: Static Web Project → public → index.html + CSS + imagen → configurar Tomcat → Start → Run As → Run on Server → abrir localhost:8080/...
P2: formulario recibe consumo → servlet lee parámetro → mpg = 235,214 / consumo → compara mpg con la tabla → muestra resultado e impuesto.
P3: añadir en la web del CV un enlace a la URL/contexto de la aplicación 2 y comprobar que funciona.

Base del contenido: documento “MP_0614 – UD1 – Examen (Modelo A)” aportado en esta conversación. Se han mantenido sus tres preguntas y sus requisitos. La fórmula de conversión se ha tomado de la expresión matemática impresa; se ha señalado aparte la discrepancia con el fragmento de código del propio documento.